

Difusão viscosa 2D: particionamento e balanceamento de carga

Eugenio Vitor Lopes dos Santos

DCA3703, Programação Paralela, Universidade Federal do Rio Grande do Norte

1 Enunciado

“Escreva um código que simule o movimento de um fluido ao longo do tempo usando a equação de Navier-Stokes, considerando apenas os efeitos da viscosidade. Desconsidere a pressão e quaisquer forças externas. Utilize diferenças finitas para discretizar o espaço e simule a evolução da velocidade do fluido no tempo. Inicialize o fluido parado ou com velocidade constante e verifique se o campo permanece estável. Em seguida, crie uma pequena perturbação e observe se ela se difunde suavemente. Após validar o código, paralelize-o com OpenMP e explore o impacto das cláusulas `schedule` e `collapse` no desempenho da execução paralela.”

2 Fundamentação teórica

2.1 Particionamento de dados e divisão de trabalho

Particionamos o domínio em partes e atribuímos cada parte às threads. No stencil desta tarefa, todas as células internas executam a mesma operação sobre os vizinhos. A divisão natural é espacial, por linhas ou por células. O equilíbrio depende do tempo de cada parte, não apenas do número de iterações. Se o custo for uniforme, uma divisão uniforme tende a funcionar bem. Custos diferentes fazem algumas threads esperar pelas outras.

Em OpenMP, `parallel` cria a equipe de threads. `for` distribui as iterações de um laço dentro dessa equipe, sem criar outra equipe. Um laço C comum dentro da região paralela é executado por todas as threads que o encontram. Se envolvêssemos o stencil apenas com `parallel`, cada thread repetiria as atualizações e o programa teria corridas.

2.2 Divisão de blocos com `sections`

A diretiva `#pragma omp sections` distribui blocos estruturados, marcados por `section`, entre as threads da equipe. Cada bloco executa uma vez, mas uma mesma thread pode receber mais de um bloco. A implementação decide a atribuição. O programador não deve presumir a ordem textual nem um ID fixo de thread. A construção inclui uma barreira no final, salvo quando usamos `nowait`.

```
1 #pragma omp parallel sections
2 {
3     #pragma omp section
4     { /* calcular um diagnostico independente */ }
5     #pragma omp section
6     { /* calcular outro diagnostico independente */ }
```

Listing 1: Exemplo conceitual de sections, não utilizado nas sete versões

`sections` divide blocos de operações. `for` divide as iterações de um laço. Duas seções permitem no máximo duas threads executando esses blocos ao mesmo tempo, mesmo que a equipe seja maior. Uma seção longa não é subdividida para ocupar threads ociosas. `schedule` também não se aplica a `sections`.

Nesta grade, uma seção para cada região fixaria a decomposição no código. `for` ajusta a distribuição ao número de threads e ao tamanho da grade. Por isso, estudamos `sections`, mas não a usamos nas variantes. Para calcular diagnósticos independentes com essa construção, o campo teria de estar pronto e cada bloco teria de escrever em um destino próprio.

2.3 A cláusula `schedule`: `static`, `dynamic` e `guided`

A cláusula `schedule(tipo, chunk)` agrupa as iterações lógicas em blocos contíguos e define como as threads recebem esses blocos. Ela não elimina dependências, não protege variáveis compartilhadas e não escolhe o núcleo onde cada thread roda. Afinidade e distribuição de iterações são decisões diferentes.

Com `schedule(static)` sem `chunk` explícito, o runtime divide as iterações em blocos aproximadamente iguais, com no máximo um bloco por thread. Os limites exatos podem variar entre implementações. A atribuição não depende da ordem em que as threads terminam. Essa política reduz o gerenciamento durante o laço e favorece acessos contíguos quando cada iteração representa uma linha da matriz.

Com `schedule(static, k)`, o runtime atribui os blocos de `k` iterações em ciclo, seguindo a ordem das threads. Para 16 iterações, quatro threads e `k=2`, a thread 0 recebe 0–1 e 8–9, enquanto a thread 1 recebe 2–3 e 10–11. As demais recebem os blocos restantes. Assim, `static` com `chunk` não garante um único intervalo contíguo por thread.

Com `schedule(dynamic, k)`, uma thread recebe um bloco e pede outro quando termina, enquanto houver blocos disponíveis. O último bloco pode ter menos de `k` iterações. Sem `chunk` explícito, `k=1`. A política ajuda quando o custo das iterações ou a velocidade das threads varia, mas cobra um despacho a cada bloco. Em uma carga uniforme, ela pode não ganhar de `static`.

Com `schedule(guided, k)`, as threads também pedem novos blocos ao terminar. Os blocos começam maiores e diminuem conforme o trabalho restante e o número de threads. `k` é o tamanho mínimo, salvo no último bloco. Não é um tamanho fixo. Sem `k` explícito, o mínimo é 1. A sequência exata depende da implementação. `Guided` reduz os despachos no começo e usa blocos menores no final, mas um bloco inicial caro ainda pode atrasar a equipe.

As versões definem as políticas diretamente nos pragmas. Alterar `OMP_SCHEDULE` não muda esses laços, pois a variável só controla construções com `schedule(runtime)`. O número de threads é outro parâmetro. O programa o recebe de `OMP_NUM_THREADS` e registra o valor observado dentro da região paralela.

2.4 Chunksize: compromisso entre overhead e equilíbrio

O chunksize conta iterações lógicas. Não conta bytes, threads ou núcleos. Blocos pequenos oferecem mais pontos de redistribuição, mas aumentam o custo de gerenciamento e podem fragmentar a localidade. Blocos grandes amortizam esse custo, porém deixam menos unidades para distribuir e podem concentrar trabalho no final.

No stencil, todas as células fazem o mesmo número de operações. A flexibilidade extra de dynamic ou guided pode custar mais do que economiza. Só os tempos podem testar essa hipótese. Diferenças entre threads, interferência externa e acessos remotos à memória também alteram o equilíbrio.

2.5 A cláusula `collapse`

`collapse(2)` junta os dois laços espaciais em um único espaço lógico, que o `schedule` distribui. Sem `collapse`, `omp for` distribui apenas o laço externo `i`. Cada unidade é uma linha com $N-2$ células internas. Com `collapse`, cada unidade é um par (i,j) , ou seja, uma célula.

Para a grade retangular desta tarefa, a linearização conceitual é

$$q = (i - 1)(N - 2) + (j - 1), \quad 0 \leq q < (N - 2)^2.$$

O compilador não precisa calcular q explicitamente a cada atualização. Os laços têm limites definidos, são independentes, usam incremento unitário e seguem a forma canônica exigida. `Collapse` não cria threads, não altera a disposição da matriz e não remove dependências de dados.

Se $N-2$ for menor que o número de threads, a divisão por linhas deixa parte da equipe ociosa. `Collapse` pode expor mais trabalho. Quando já há muitas linhas, o ganho não é automático. É preciso considerar a nova granularidade, a geração de índices e a vetorização. O laço temporal fica fora da distribuição porque cada passo depende do campo anterior.

2.6 Paralelismo vetorial com `simd`

A diretiva `#pragma omp simd` permite executar várias iterações com instruções SIMD. Uma instrução SIMD opera sobre vários elementos e usa a unidade vetorial de um núcleo. Ela não cria threads. `omp for` distribui trabalho entre threads. As duas formas podem ser combinadas.

Uma opção seria distribuir as linhas com `omp for` e aplicar `omp simd` ao laço interno `j`. Cada thread cuidaria de suas linhas e o núcleo processaria células contíguas em vetores. `omp for simd` combina as duas construções no mesmo espaço de iterações.

Os buffers separados tornam as atualizações espaciais independentes. Ler um vizinho do campo antigo não interfere na escrita de outra célula do campo novo. Essa propriedade favorece a vetorização. Se o programa atualizasse o mesmo buffer, a análise seria outra. Também não declaramos alinhamento de 64 bytes sem garanti-lo, pois `malloc` não oferece necessariamente esse alinhamento.

Nenhuma das sete versões contém uma diretiva `simd` explícita. O compilador ainda pode autovetorizar. A largura efetiva depende do alvo e das opções de compilação. Para medir esse

efeito, criaríamos uma variante com `simd`, manteríamos as demais condições e consultaríamos o relatório de vetorização do compilador. No GCC, estas opções mostram otimizações realizadas e oportunidades perdidas:

```
1 -fopt-info-vec-optimized -fopt-info-vec-missed
```

Listing 2: Opções de diagnóstico de vetorização para um experimento adicional

A ausência do pragma não prova que não houve SIMD. A presença do pragma também não prova que o código ficou mais rápido.

2.7 Dependências, barreiras e custo paralelo

A correção vem antes da escolha da política de distribuição. Em cada passo, todas as threads leem o buffer antigo e cada célula do buffer novo tem um único escritor. Entre passos há uma dependência RAW. O próximo passo precisa dos valores produzidos pelo atual. A barreira no final de `omp for` deixa o campo pronto antes da troca dos ponteiros.

A troca ocorre em `single`, executado por uma thread da equipe. Ela não precisa ser a thread principal. A barreira final faz todas as threads prosseguirem com os ponteiros já trocados. Sem a primeira barreira, a troca poderia ocorrer durante escritas pendentes. Sem a segunda, o próximo passo poderia começar antes de todas as threads verem os novos ponteiros. Um `critical` por célula não é necessário. Ele serializaria atualizações independentes.

O tempo paralelo reúne trabalho útil, distribuição dos blocos, espera por desbalanceamento, sincronização e acesso à memória. Para p threads, calculamos o speedup $S(p) = T_{seq}/T_p$ e a eficiência $E(p) = S(p)/p$. O caso ideal $S(p)=p$ exige divisão perfeita e nenhum custo extra. Uma política pode equilibrar melhor o trabalho e ainda perder tempo no gerenciamento.

2.8 Modelo numérico usado como carga de trabalho

No modelo usado, “apenas efeitos da viscosidade” também significa omitir a advecção, a pressão e as forças externas. Resolvemos uma componente escalar da velocidade, não o sistema completo de Navier–Stokes incompressível. Para viscosidade constante, $\partial_t u = \nu \nabla^2 u$.

O esquema FTCS em uma grade quadrada usa

$$u_{i,j}^{n+1} = u_{i,j}^n + r(u_{i-1,j}^n + u_{i+1,j}^n + u_{i,j-1}^n + u_{i,j+1}^n - 4u_{i,j}^n), \quad r = \frac{\nu \Delta t}{h^2}.$$

Usamos $\nu = 0.1$, $\Delta t = 0.2$ e $h = 1$, logo $r = 0.02$. Em duas dimensões, o esquema é estável quando $0 \leq r \leq 1/4$. Para uma solução suave e uma integração estável, o erro global é $O(\Delta t + h^2)$.

Nos modos parado e constante, as bordas têm o mesmo valor do interior, zero e um, respectivamente. A perturbação é uma gaussiana de amplitude 0.1 e $\sigma = N/10$, com bordas nulas desde o início. Nesse modo, a soma pode diminuir porque a difusão leva valores até as bordas.

3 Implementação

3.1 Organização das versões e sincronização

Cada arquivo `v*.c` é um programa completo e independente. A duplicação facilita a comparação dos pragmas. O tamanho da grade e o número de passos são constantes nomeadas diretamente no código como `tamanho_grid=512` e `num_passos_tempo=500`. Cada programa recebe apenas o modo inicial e um caminho opcional para exportar o campo. Ele valida esses argumentos e o retorno de `malloc`. Dois buffers separam a leitura do instante antigo da escrita do novo. As bordas são inicializadas nos dois buffers e permanecem fixas durante o laço temporal, sem escritas concorrentes nos cantos.

A versão sequencial compila sem OpenMP. As seis variantes paralelas comparam `static`, `dynamic` com `chunk 16` e `guided` com mínimo 16. Cada política aparece com e sem `collapse(2)`. Uma única região paralela contém todos os passos, então o programa não reentra em uma região paralela a cada passo. As barreiras continuam necessárias. O runtime também pode reutilizar as threads, em vez de criar threads do sistema operacional a cada região.

`default(none)` exige que o código declare os escopos das variáveis. Os ponteiros `u` e `v` são compartilhados, assim como `tamanho_grid`, `num_passos_tempo`, `r` e o registro do número real de threads. Compartilhar um ponteiro não duplica o campo apontado. Os contadores declarados dentro da região e as variáveis locais `c` e `tmp` pertencem a cada thread. Tornar apenas os ponteiros `private` não criaria uma matriz privada inicializada.

Cada thread percorre o laço temporal com seu contador `t` local. Todas encontram os laços de distribuição na mesma ordem. O `omp for` atribui cada célula a uma única thread em cada passo. Dentro de `single`, `tmp` troca `u` e `v` sem copiar a matriz. A ausência de corridas vem da exclusividade das escritas espaciais e das duas barreiras. `schedule` só define a distribuição do trabalho.

Versão	Política	Unidade distribuída
v0	Sequencial	Sem distribuição
v1	<code>static</code>	Linha
v2	<code>static, collapse(2)</code>	Célula
v3	<code>dynamic, 16</code>	Bloco de 16 linhas
v4	<code>dynamic, 16, collapse(2)</code>	Bloco de 16 células
v5	<code>guided, 16, collapse(2)</code>	Bloco decrescente de células
v6	<code>guided, 16</code>	Bloco decrescente de linhas

Table 1: Versões completas e independentes. Em `guided`, 16 é o tamanho mínimo, salvo o último bloco.

3.2 Compilação e execução

```
1 gcc -std=c11 -O2 -Wall -Wextra v0_seq.c -lm -o ns_v0
2 gcc -std=c11 -O2 -Wall -Wextra -fopenmp v1_static.c -lm -o ns_v1
3 ./ns_v0 1
4 ./ns_v0 2
5 OMP_NUM_THREADS=4 ./ns_v1 0 campo.txt
```

Listing 3: Compilação e execução a partir de `tarefa11/`

4 Resultados

4.1 Validação numérica

Compilamos as sete variantes com GCC e as opções `-std=c11`, `-O2`, `-Wall`, `-Wextra` e `-Werror`. Usamos `-fopenmp` apenas nas versões paralelas. Os testes aprovaram 30 comparações de campos completos de 512×512 entre as variantes. Foram verificados os três modos iniciais e, para a perturbação, execuções com 1, 2 e 4 threads. O campo parado permaneceu nulo, o campo constante permaneceu unitário e todas as versões paralelas produziram exatamente o mesmo campo da referência sequencial. Também verificamos a queda do pico gaussiano e a rejeição de entradas inválidas.

A Figura 1 mostra a perturbação gaussiana antes e depois dos 500 passos. O pico diminui enquanto o raio quadrático médio aumenta, sem oscilações nem novos extremos, comportamento esperado para difusão viscosa estável.

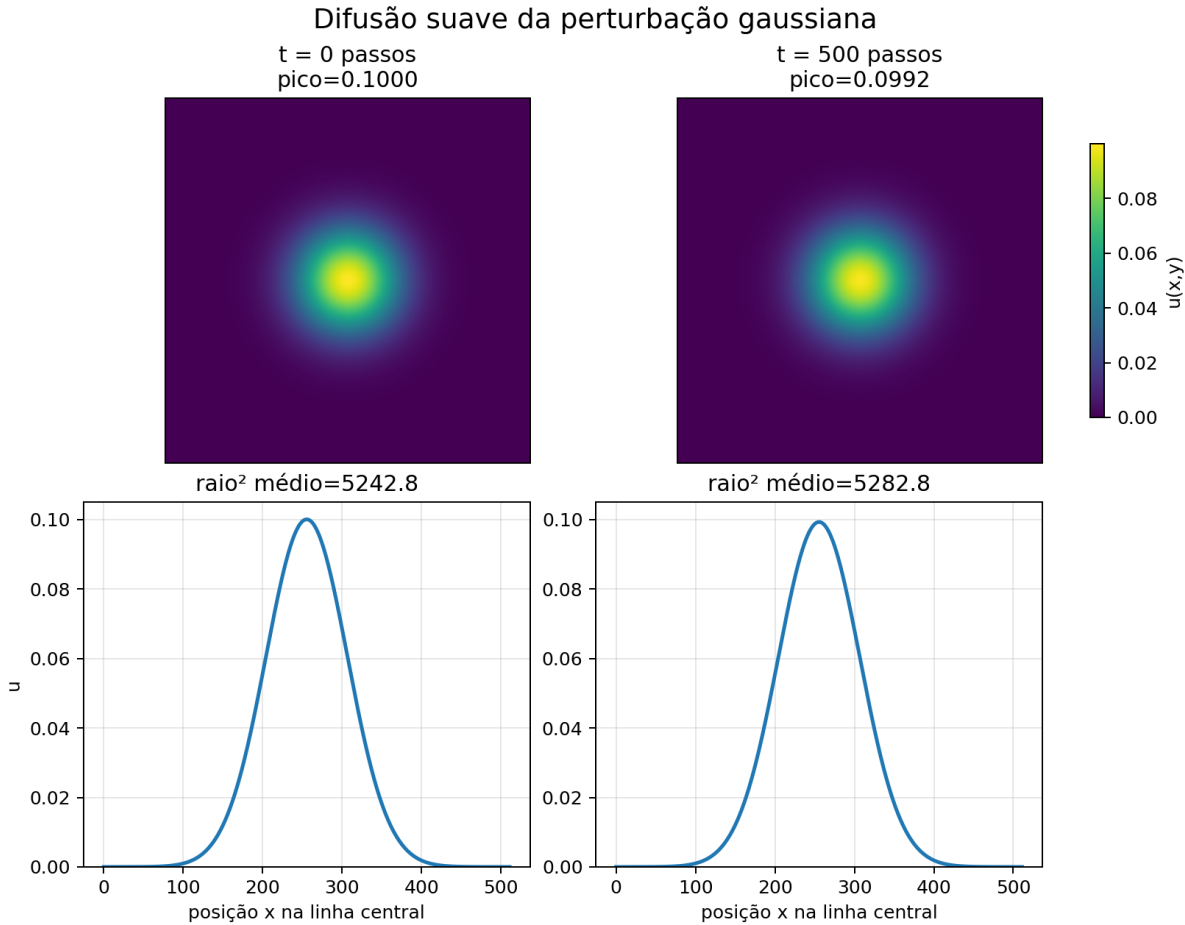


Figure 1: Difusão da perturbação gaussiana no início e após 500 passos.

4.2 Medição de desempenho

O benchmark foi executado com GCC 13.3.0 no Intel Core i7-14700HX sob WSL2, usando uma grade 512×512 , 500 passos e todas as quantidades de 1 a 28 threads disponíveis. Cada ponto de

1 a 25 threads usa a mediana de cinco repetições após aquecimento. Como a primeira varredura apresentou picos abruptos no final, os pontos de 26 a 28 threads foram repetidos dez vezes sob as mesmas condições e substituídos por essas medianas confirmadas. Foram usadas as opções `-std=c11`, `-O2`, `-Wall` e `-Wextra`, além de `-lm`; nas variantes paralelas, acrescentamos `-fopenmp`. Fixamos `OMP_DYNAMIC=FALSE`, `OMP_PROC_BIND=close` e `OMP_PLACES=cores`.

O cronômetro `CLOCK_MONOTONIC` cobre a região paralela, o stencil, as barreiras e a troca de ponteiros. Ele não cobre alocação, inicialização ou exportação. A Tabela 2 apresenta medianas selecionadas; as Figuras 2 e 3 mostram toda a varredura e o speedup $S(p) = T_{seq}/T_p$. O conjunto consolidado está em `runs/resultado-confirmado/`; as duas execuções de origem permanecem preservadas para rastreabilidade.

Versão	1 thread	2 threads	4 threads	8 threads	14 threads	28 threads
v1 static	0,0970	0,0531	0,0289	0,0223	0,0207	0,0185
v2 static + collapse	0,1312	0,0722	0,0411	0,0326	0,0280	0,0488
v3 dynamic	0,1064	0,0621	0,0337	0,0220	0,0228	0,0276
v4 dynamic + collapse	0,2241	0,3662	0,3482	0,3342	0,3007	0,3225
v5 guided + collapse	0,1318	0,0706	0,0395	0,0277	0,0242	0,0329
v6 guided	0,1239	0,0588	0,0327	0,0240	0,0229	0,0269

Table 2: Tempo mediano em segundos, selecionado da varredura completa de 1 a 28 threads. A referência sequencial teve mediana de 0,1049 s.

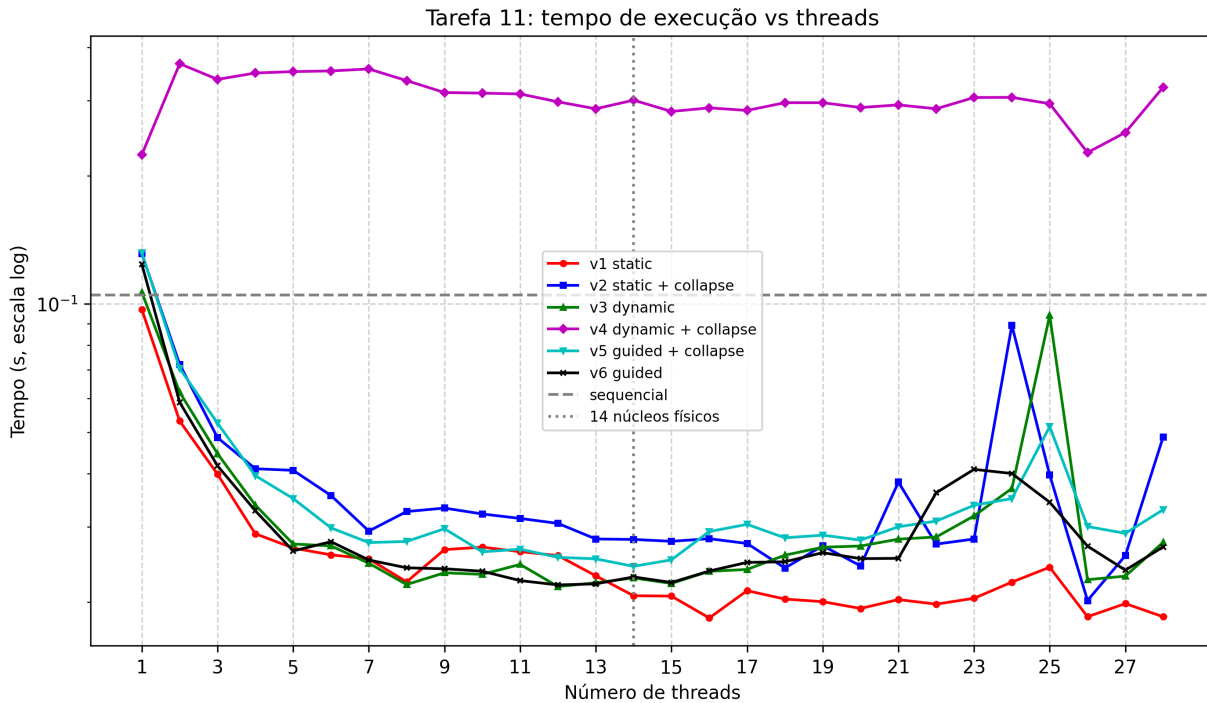


Figure 2: Tempo mediano das políticas com e sem `collapse(2)`, de 1 a 28 threads, em escala logarítmica.

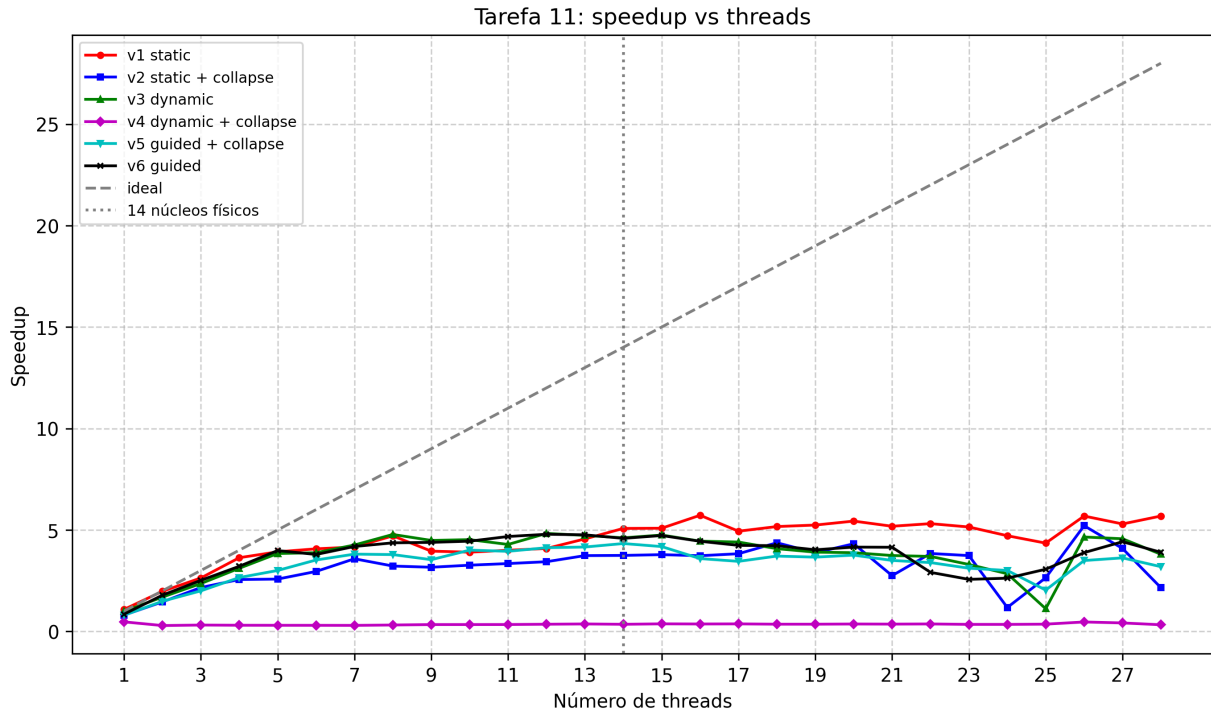


Figure 3: Speedup relativo à referência sequencial. A reta tracejada representa o caso ideal.

4.3 Variação do chunksize

Para separar o efeito de `collapse` do tamanho físico de cada bloco, realizamos um segundo experimento com oito threads. Sem `collapse`, os chunks foram 1, 4, 16 e 64 linhas. Com `collapse`, usamos 510, 2.040, 8.160 e 32.640 células, quantidades equivalentes a 1, 4, 16 e 64 linhas internas. Cada configuração teve cinco repetições; os dados estão em `runs/chunks-20260911-152008-206654/`.

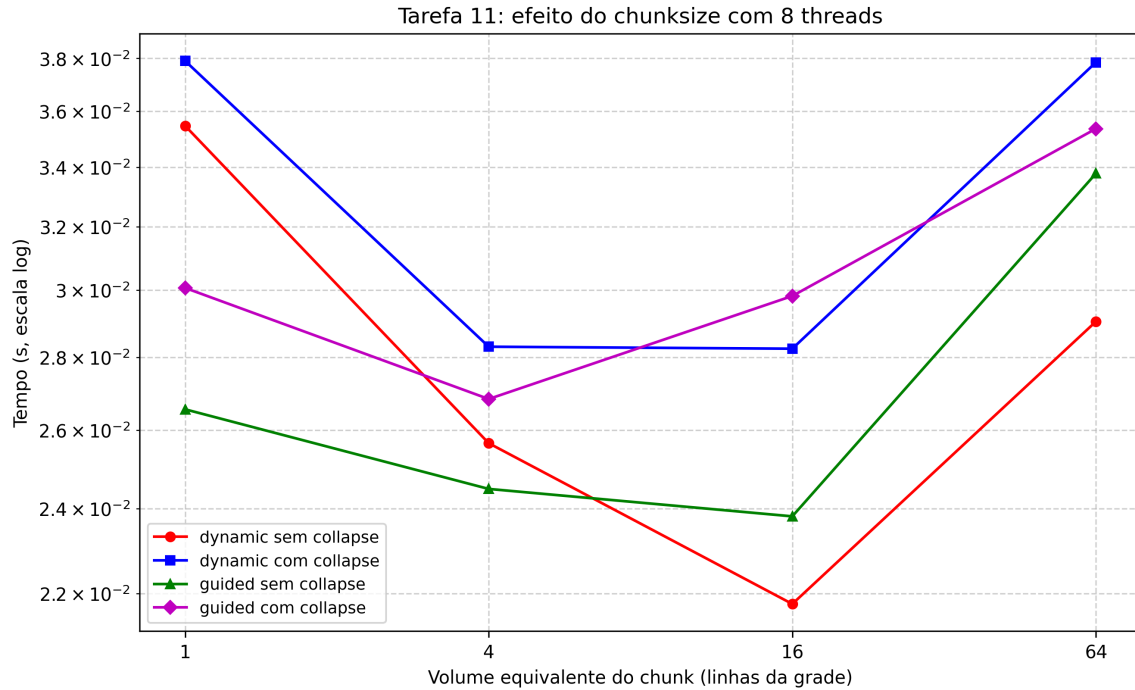


Figure 4: Efeito do chunksize com oito threads. O eixo horizontal representa o volume equivalente em linhas.

No *dynamic*, o melhor resultado sem collapse ocorreu com 16 linhas por bloco (0,0218 s); com collapse, quatro e 16 linhas equivalentes ficaram próximos (0,0283 s). No *guided*, quatro linhas foi o melhor compromisso sem collapse (0,0245 s), enquanto uma linha equivalente foi melhor com collapse (0,0301 s). Chunks muito pequenos ampliaram o despacho; chunks de 64 linhas reduziram as oportunidades de balanceamento.

5 Discussão

5.1 Granularidade e balanceamento

Sem collapse, chunk 16 significa 16 linhas. Com collapse, significa 16 células. Para $N=512$, a grade tem 510 linhas internas e 260.100 células internas. Portanto, *dynamic* com chunk 16 distribui $\lceil 510/16 \rceil = 32$ blocos por passo sem collapse e $\lceil 260100/16 \rceil = 16.257$ com collapse. Esses números são blocos do laço, não threads ou tarefas novas.

No primeiro caso, cada bloco completo contém 8.160 atualizações. No segundo, contém apenas 16. Comparar *dynamic*, 16 por linhas com *dynamic*, 8160 por células aproximaria o volume de trabalho por bloco. Esse controle separa o custo de collapse do custo da granularidade. Em *guided*, o chunk é um mínimo, então a quantidade de blocos não segue essa divisão fixa.

Comparamos v1 com v2, v3 com v4 e v6 com v5 para estudar collapse dentro de cada política. A comparação entre v1, v3 e v6 mantém a distribuição por linhas. A comparação entre v2, v4 e v5 mantém a distribuição por células. Para comparar tempos, mantemos N , o número de passos, o

modo inicial, o compilador e a equipe. Resultados numéricos iguais confirmam os casos testados, mas não mostram que o trabalho foi distribuído de modo igual.

5.2 Localidade e limites de escalabilidade

Com quatro threads, `static` sem `collapse` obteve o menor tempo: 0,0289 s e speedup 3,63. Na varredura completa, o menor tempo global foi 0,0183 s com `static` sem `collapse` e 16 threads, correspondente a speedup 5,72. `dynamic` e `guided` sem `collapse` atingiram seus menores tempos com 12 threads, respectivamente 0,0217 s e 0,0219 s. A carga uniforme favorece a política estática, que distribui as linhas com pouco gerenciamento. `C` armazena a matriz em ordem row-major; variar `j` acessa posições contíguas e reutiliza vizinhos no cache, e blocos de linhas preservam essa localidade.

O impacto de `collapse(2)` foi predominantemente negativo nesta configuração. Até mesmo `static` com `collapse` passou para 0,0411 s com quatro threads. O pior caso foi `dynamic`, 16 com `collapse`: 0,3482 s e speedup 0,30. Sem `collapse`, cada chunk reúne 16 linhas; com `collapse`, reúne somente 16 células. Para $N = 512$, isso aumenta de 32 para aproximadamente 16.257 blocos por passo. O custo de milhões de despachos ao longo dos 500 passos supera o trabalho útil. `guided` reduz esse problema usando blocos inicialmente maiores, mas sua versão com `collapse` ainda foi mais lenta que a divisão por linhas.

Em uma linha de cache de 64 bytes cabem oito doubles. Threads que escrevem células distintas na mesma linha podem provocar falso compartilhamento, mesmo sem disputar a mesma célula. O efeito depende dos limites dos blocos e do alinhamento. Chunk 16 não o elimina em todos os limites de linha da matriz.

Cache, banda de memória, barreiras e afinidade podem limitar a escala. Os dois buffers da grade 512×512 ocupam 4 MiB e podem caber em cache. Por isso, speedup baixo não prova saturação da RAM. Após aproximadamente 16 threads, a versão `static` sem `collapse` entrou em um platô: 0,0183 s com 16 threads e 0,0185 s com 28. As demais políticas deixaram de melhorar antes disso. Os picos inicialmente observados em 26–28 threads não se repetiram de forma sistemática no ensaio de confirmação, mostrando a importância de repetir medições em ambiente virtualizado. Ainda assim, barreiras, gerenciamento OpenMP, SMT e os núcleos híbridos do processador limitam o ganho nessa carga curta.

5.3 Papel de sections e simd nesta aplicação

Não usamos `sections` porque o trabalho principal é um conjunto grande de atualizações iguais, não poucos blocos de funções diferentes. SIMD atua dentro do núcleo e pode complementar a divisão entre threads. Como não incluímos uma variante `simd` nem medimos a vetorização, os resultados não permitem atribuir ganhos a essa diretiva. Essa distinção separa os conceitos discutidos das técnicas avaliadas.

6 Conclusão

Os testes preservaram os campos zero e unitário e verificaram a difusão da perturbação sem criar novos extremos. As sete versões produziram campos idênticos nos casos testados. A região paralela persistente mantém as barreiras necessárias para atualizar o stencil e trocar os buffers.

A comparação entre políticas precisa considerar a unidade do chunk. Sem collapse, ela é uma linha; com collapse, é uma célula. Para esta carga uniforme e para a grade avaliada, `schedule(static)` sem `collapse` apresentou o melhor desempenho com quatro threads e também o menor tempo de toda a varredura, com 16 threads. O ganho saturou antes do máximo de 28 threads. A versão `dynamic, 16` com collapse foi a menos adequada porque sua granularidade fina gerou overhead de distribuição muito superior ao trabalho de cada bloco.

A Código-fonte

Apresentamos os arquivos completos para comparar diretamente as alterações de OpenMP.

```
1 // gcc -std=c11 -O2 -Wall -Wextra v0_seq.c -lm -o ns_v0
2 #ifdef _OPENMP
3 #error "Compile a referencia sequencial sem -fopenmp"
4 #endif
5 #define _POSIX_C_SOURCE 199309L
6 #include <errno.h>
7 #include <math.h>
8 #include <stdio.h>
9 #include <stdlib.h>
10 #include <string.h>
11 #include <time.h>
12
13 static int integer(const char *text, int minimum, int maximum, int *out) {
14     char *end;
15     errno = 0;
16     long value = strtol(text, &end, 10);
17     if (errno || end == text || *end || value < minimum || value > maximum
18         )
19         return 0;
20     *out = (int)value;
21     return 1;
22 }
23
24 static double agora(void) {
25     struct timespec t;
26     if (clock_gettime(CLOCK_MONOTONIC, &t)) {
27         perror("clock_gettime");
28         exit(EXIT_FAILURE);
29     }
30     return (double)t.tv_sec + (double)t.tv_nsec / 1e9;
31 }
32
33 int main(int argc, char **argv) {
34     const int tamanho_grid = 512;
35     const int num_passos_tempo = 500;
36     int mode = 0, threads = 1;
37     if (argc > 3 || (argc > 1 && !integer(argv[1], 0, 2, &mode))) {
38         fprintf(stderr, "Uso: %s [modo=0|1|2 [campo.txt]]\n", argv[0]);
39         return EXIT_FAILURE;
40     }
41     size_t count = (size_t)tamanho_grid * tamanho_grid;
42     double *u = malloc(count * sizeof(*u));
43     double *v = malloc(count * sizeof(*v));
44     if (!u || !v) {
45         fprintf(stderr, "Falha de alocacao\n");
46     }
```

```

45     free(u); free(v);
46     return EXIT_FAILURE;
47 }
48 const double nu = 0.1, dt = 0.2, dx = 1.0;
49 const double r = nu * dt / (dx * dx); /* 0.02 <= 1/4 em 2D */
50 for (int i = 0; i < tamanho_grid; ++i) {
51     for (int j = 0; j < tamanho_grid; ++j) {
52         double x = i - (tamanho_grid - 1) / 2.0, y = j - (tamanho_grid
53             - 1) / 2.0;
54         double sigma = tamanho_grid / 10.0;
55         int boundary = i == 0 || j == 0 || i == tamanho_grid-1 || j ==
56             tamanho_grid-1;
57         u[(size_t)i*tamanho_grid+j] = mode == 2 ? 1.0 :
58             (mode == 1 || boundary ? 0.0 :
59                 0.1 * exp(-(x*x+y*y)/(2*sigma*sigma)));
60     }
61 }
62 /* Bordas fixas iguais nos dois buffers: nenhuma escrita posterior. */
63 memcpy(v, u, count * sizeof(*v));
64 double start = agora();
65 {
66     for (int t = 0; t < num_passos_tempo; ++t) {
67         for (int i = 1; i < tamanho_grid-1; ++i) {
68             for (int j = 1; j < tamanho_grid-1; ++j) {
69                 size_t c = (size_t)i*tamanho_grid+j;
70                 v[c] = u[c] + r * (u[c-1]+u[c+1]+u[c-tamanho_grid]+u[c
71                     +tamanho_grid]-4*u[c]);
72             }
73         } /* Barreira: todas as escritas terminam antes da troca. */
74         {
75             double *tmp = u; u = v; v = tmp;
76         } /* Barreira: todas as threads veem a troca antes do próximo
77             passo. */
78     }
79 }
80 double elapsed = agora() - start;
81 double peak = u[0], minimum = u[0], sum = 0.0;
82 for (size_t c = 0; c < count; ++c) {
83     peak = fmax(peak, u[c]); minimum = fmin(minimum, u[c]); sum += u[c
84 ];
85 }
86 printf("%s tamanho_grid=%d num_passos_tempo=%d modo=%d threads=%d r
87     =%.3f tempo=%.9f s minimo=%.17g pico=%.17g soma=%.17g\n",
88     "v0_seq", tamanho_grid, num_passos_tempo, mode, threads, r,
89     elapsed, minimum, peak, sum);
90 int status = EXIT_SUCCESS;
91 if (argc == 3) {
92     FILE *file = fopen(argv[2], "w");
93     if (!file) { perror(argv[2]); status = EXIT_FAILURE; }

```

```

87     else {
88         for (size_t c = 0; c < count; ++c)
89             if (fprintf(file, "%.17g%c", u[c], (c+1)%tamanho_grid ? '
90                 ' : '\n') < 0)
91                 status = EXIT_FAILURE;
92             if (fclose(file)) status = EXIT_FAILURE;
93         }
94     }
95     free(u); free(v);
96     return status;
97 }

```

Listing 4: v0_seq.c: referência sequencial

```

1 // gcc -std=c11 -O2 -Wall -Wextra -fopenmp vi_static.c -lm -o ns_vi
2 #ifndef _OPENMP
3 #error "Compile esta variante com -fopenmp"
4 #endif
5 #define _POSIX_C_SOURCE 199309L
6 #include <errno.h>
7 #include <math.h>
8 #include <stdio.h>
9 #include <stdlib.h>
10 #include <string.h>
11 #include <time.h>
12 #include <omp.h>
13
14 static int integer(const char *text, int minimum, int maximum, int *out) {
15     char *end;
16     errno = 0;
17     long value = strtol(text, &end, 10);
18     if (errno || end == text || *end || value < minimum || value > maximum
19         )
20         return 0;
21     *out = (int)value;
22     return 1;
23 }
24
25 static double agora(void) {
26     struct timespec t;
27     if (clock_gettime(CLOCK_MONOTONIC, &t)) {
28         perror("clock_gettime");
29         exit(EXIT_FAILURE);
30     }
31     return (double)t.tv_sec + (double)t.tv_nsec / 1e9;
32 }
33
34 int main(int argc, char **argv) {
35     const int tamanho_grid = 512;
36     const int num_passos_tempo = 500;
37     int mode = 0, threads = 1;
38     if (argc > 3 || (argc > 1 && !integer(argv[1], 0, 2, &mode))) {
39         fprintf(stderr, "Uso: %s [modo=0|1|2 [campo.txt]]\n", argv[0]);
40         return EXIT_FAILURE;
41     }
42     size_t count = (size_t)tamanho_grid * tamanho_grid;
43     double *u = malloc(count * sizeof(*u));
44     double *v = malloc(count * sizeof(*v));
45     if (!u || !v) {
46         fprintf(stderr, "Falha de alocao\n");
47         free(u); free(v);
48         return EXIT_FAILURE;
49     }

```

```

48 }
49 const double nu = 0.1, dt = 0.2, dx = 1.0;
50 const double r = nu * dt / (dx * dx); /* 0.02 <= 1/4 em 2D */
51 for (int i = 0; i < tamanho_grid; ++i) {
52     for (int j = 0; j < tamanho_grid; ++j) {
53         double x = i - (tamanho_grid - 1) / 2.0, y = j - (tamanho_grid
54             - 1) / 2.0;
55         double sigma = tamanho_grid / 10.0;
56         int boundary = i == 0 || j == 0 || i == tamanho_grid-1 || j ==
57             tamanho_grid-1;
58         u[(size_t)i*tamanho_grid+j] = mode == 2 ? 1.0 :
59             (mode == 1 || boundary ? 0.0 :
60                 0.1 * exp(-(x*x+y*y)/(2*sigma*sigma)));
61     }
62 }
63 /* Bordas fixas iguais nos dois buffers: nenhuma escrita posterior. */
64 memcpy(v, u, count * sizeof(*v));
65 double start = agora();
66 #pragma omp parallel default(none) shared(u,v,tamanho_grid,
67     num_passos_tempo,r,threads)
68 {
69     #pragma omp single
70     threads = omp_get_num_threads();
71     for (int t = 0; t < num_passos_tempo; ++t) {
72         #pragma omp for schedule(static)
73         for (int i = 1; i < tamanho_grid-1; ++i) {
74             for (int j = 1; j < tamanho_grid-1; ++j) {
75                 size_t c = (size_t)i*tamanho_grid+j;
76                 v[c] = u[c] + r * (u[c-1]+u[c+1]+u[c-tamanho_grid]+u[c
77                     +tamanho_grid]-4*u[c]);
78             }
79         } /* Barreira: todas as escritas terminam antes da troca. */
80         #pragma omp single
81         {
82             double *tmp = u; u = v; v = tmp;
83         } /* Barreira: todas as threads veem a troca antes do próximo
84             passo. */
85     }
86 }
87 double elapsed = agora() - start;
88 double peak = u[0], minimum = u[0], sum = 0.0;
89 for (size_t c = 0; c < count; ++c) {
90     peak = fmax(peak, u[c]); minimum = fmin(minimum, u[c]); sum += u[c]
91 ];
92 }
93 printf("%s tamanho_grid=%d num_passos_tempo=%d modo=%d threads=%d r
94     =%.3f tempo=%.9f s minimo=%.17g pico=%.17g soma=%.17g\n",
95     "v1_static", tamanho_grid, num_passos_tempo, mode, threads, r,
96     elapsed, minimum, peak, sum);

```

```

89     int status = EXIT_SUCCESS;
90     if (argc == 3) {
91         FILE *file = fopen(argv[2], "w");
92         if (!file) { perror(argv[2]); status = EXIT_FAILURE; }
93         else {
94             for (size_t c = 0; c < count; ++c)
95                 if (fprintf(file, "%.17g%c", u[c], (c+1)%tamanho_grid ? '
96                     ' : '\n') < 0)
97                     status = EXIT_FAILURE;
98             if (fclose(file)) status = EXIT_FAILURE;
99         }
100     }
101     free(u); free(v);
102     return status;
103 }

```

Listing 5: v1_static.c: static por linhas

```

1 // gcc -std=c11 -O2 -Wall -Wextra -fopenmp v2_static_collapse.c -lm -o
   ns_v2
2 #ifndef _OPENMP
3 #error "Compile esta variante com -fopenmp"
4 #endif
5 #define _POSIX_C_SOURCE 199309L
6 #include <errno.h>
7 #include <math.h>
8 #include <stdio.h>
9 #include <stdlib.h>
10 #include <string.h>
11 #include <time.h>
12 #include <omp.h>
13
14 static int integer(const char *text, int minimum, int maximum, int *out) {
15     char *end;
16     errno = 0;
17     long value = strtol(text, &end, 10);
18     if (errno || end == text || *end || value < minimum || value > maximum
19         )
20         return 0;
21     *out = (int)value;
22     return 1;
23 }
24
25 static double agora(void) {
26     struct timespec t;
27     if (clock_gettime(CLOCK_MONOTONIC, &t)) {
28         perror("clock_gettime");
29         exit(EXIT_FAILURE);
30     }
31     return (double)t.tv_sec + (double)t.tv_nsec / 1e9;
32 }
33
34 int main(int argc, char **argv) {
35     const int tamanho_grid = 512;
36     const int num_passos_tempo = 500;
37     int mode = 0, threads = 1;
38     if (argc > 3 || (argc > 1 && !integer(argv[1], 0, 2, &mode))) {
39         fprintf(stderr, "Uso: %s [modo=0|1|2 [campo.txt]]\n", argv[0]);
40         return EXIT_FAILURE;
41     }
42     size_t count = (size_t)tamanho_grid * tamanho_grid;
43     double *u = malloc(count * sizeof(*u));
44     double *v = malloc(count * sizeof(*v));
45     if (!u || !v) {
46         fprintf(stderr, "Falha de alocao\n");
47         free(u); free(v);

```

```

47     return EXIT_FAILURE;
48 }
49 const double nu = 0.1, dt = 0.2, dx = 1.0;
50 const double r = nu * dt / (dx * dx); /* 0.02 <= 1/4 em 2D */
51 for (int i = 0; i < tamanho_grid; ++i) {
52     for (int j = 0; j < tamanho_grid; ++j) {
53         double x = i - (tamanho_grid - 1) / 2.0, y = j - (tamanho_grid
54             - 1) / 2.0;
55         double sigma = tamanho_grid / 10.0;
56         int boundary = i == 0 || j == 0 || i == tamanho_grid-1 || j ==
57             tamanho_grid-1;
58         u[(size_t)i*tamanho_grid+j] = mode == 2 ? 1.0 :
59             (mode == 1 || boundary ? 0.0 :
60                 0.1 * exp(-(x*x+y*y)/(2*sigma*sigma)));
61     }
62 }
63 /* Bordas fixas iguais nos dois buffers: nenhuma escrita posterior. */
64 memcpy(v, u, count * sizeof(*v));
65 double start = agora();
66 #pragma omp parallel default(none) shared(u,v,tamanho_grid,
67     num_passos_tempo,r,threads)
68 {
69     #pragma omp single
70     threads = omp_get_num_threads();
71     for (int t = 0; t < num_passos_tempo; ++t) {
72         #pragma omp for collapse(2) schedule(static)
73         for (int i = 1; i < tamanho_grid-1; ++i) {
74             for (int j = 1; j < tamanho_grid-1; ++j) {
75                 size_t c = (size_t)i*tamanho_grid+j;
76                 v[c] = u[c] + r * (u[c-1]+u[c+1]+u[c-tamanho_grid]+u[c
77                     +tamanho_grid]-4*u[c]);
78             }
79         } /* Barreira: todas as escritas terminam antes da troca. */
80         #pragma omp single
81         {
82             double *tmp = u; u = v; v = tmp;
83         } /* Barreira: todas as threads veem a troca antes do próximo
84             passo. */
85     }
86 }
87 double elapsed = agora() - start;
88 double peak = u[0], minimum = u[0], sum = 0.0;
89 for (size_t c = 0; c < count; ++c) {
90     peak = fmax(peak, u[c]); minimum = fmin(minimum, u[c]); sum += u[c]
91 ];
92 }
93 printf("%s tamanho_grid=%d num_passos_tempo=%d modo=%d threads=%d r
94     =%.3f tempo=%.9f s minimo=%.17g pico=%.17g soma=%.17g\n",

```

```

88         "v2_static_collapse", tamanho_grid, num_passos_tempo, mode,
           threads, r, elapsed, minimum, peak, sum);
89     int status = EXIT_SUCCESS;
90     if (argc == 3) {
91         FILE *file = fopen(argv[2], "w");
92         if (!file) { perror(argv[2]); status = EXIT_FAILURE; }
93         else {
94             for (size_t c = 0; c < count; ++c)
95                 if (fprintf(file, "%.17g%c", u[c], (c+1)%tamanho_grid ? '
', : '\n') < 0)
96                     status = EXIT_FAILURE;
97             if (fclose(file)) status = EXIT_FAILURE;
98         }
99     }
100     free(u); free(v);
101     return status;
102 }

```

Listing 6: v2_static_collapse.c: static por células

```

1 // gcc -std=c11 -O2 -Wall -Wextra -fopenmp v3_dynamic.c -lm -o ns_v3
2 #ifndef _OPENMP
3 #error "Compile esta variante com -fopenmp"
4 #endif
5 #define _POSIX_C_SOURCE 199309L
6 #ifndef CHUNK_SIZE
7 #define CHUNK_SIZE 16
8 #endif
9 #include <errno.h>
10 #include <math.h>
11 #include <stdio.h>
12 #include <stdlib.h>
13 #include <string.h>
14 #include <time.h>
15 #include <omp.h>
16
17 static int integer(const char *text, int minimum, int maximum, int *out) {
18     char *end;
19     errno = 0;
20     long value = strtol(text, &end, 10);
21     if (errno || end == text || *end || value < minimum || value > maximum
22         )
23         return 0;
24     *out = (int)value;
25     return 1;
26 }
27
28 static double agora(void) {
29     struct timespec t;
30     if (clock_gettime(CLOCK_MONOTONIC, &t)) {
31         perror("clock_gettime");
32         exit(EXIT_FAILURE);
33     }
34     return (double)t.tv_sec + (double)t.tv_nsec / 1e9;
35 }
36
37 int main(int argc, char **argv) {
38     const int tamanho_grid = 512;
39     const int num_passos_tempo = 500;
40     int mode = 0, threads = 1;
41     if (argc > 3 || (argc > 1 && !integer(argv[1], 0, 2, &mode))) {
42         fprintf(stderr, "Uso: %s [modo=0|1|2 [campo.txt]]\n", argv[0]);
43         return EXIT_FAILURE;
44     }
45     size_t count = (size_t)tamanho_grid * tamanho_grid;
46     double *u = malloc(count * sizeof(*u));
47     double *v = malloc(count * sizeof(*v));
48     if (!u || !v) {

```

```

48     fprintf(stderr, "Falha de alocação\n");
49     free(u); free(v);
50     return EXIT_FAILURE;
51 }
52 const double nu = 0.1, dt = 0.2, dx = 1.0;
53 const double r = nu * dt / (dx * dx); /* 0.02 <= 1/4 em 2D */
54 for (int i = 0; i < tamanho_grid; ++i) {
55     for (int j = 0; j < tamanho_grid; ++j) {
56         double x = i - (tamanho_grid - 1) / 2.0, y = j - (tamanho_grid
57             - 1) / 2.0;
58         double sigma = tamanho_grid / 10.0;
59         int boundary = i == 0 || j == 0 || i == tamanho_grid-1 || j ==
60             tamanho_grid-1;
61         u[(size_t)i*tamanho_grid+j] = mode == 2 ? 1.0 :
62             (mode == 1 || boundary ? 0.0 :
63                 0.1 * exp(-(x*x+y*y)/(2*sigma*sigma)));
64     }
65 }
66 /* Bordas fixas iguais nos dois buffers: nenhuma escrita posterior. */
67 memcpy(v, u, count * sizeof(*v));
68 double start = agora();
69 #pragma omp parallel default(none) shared(u,v,tamanho_grid,
70     num_passos_tempo,r,threads)
71 {
72     #pragma omp single
73     threads = omp_get_num_threads();
74     for (int t = 0; t < num_passos_tempo; ++t) {
75         #pragma omp for schedule(dynamic, CHUNK_SIZE)
76         for (int i = 1; i < tamanho_grid-1; ++i) {
77             for (int j = 1; j < tamanho_grid-1; ++j) {
78                 size_t c = (size_t)i*tamanho_grid+j;
79                 v[c] = u[c] + r * (u[c-1]+u[c+1]+u[c-tamanho_grid]+u[c
80                     +tamanho_grid]-4*u[c]);
81             }
82         } /* Barreira: todas as escritas terminam antes da troca. */
83         #pragma omp single
84         {
85             double *tmp = u; u = v; v = tmp;
86         } /* Barreira: todas as threads veem a troca antes do próximo
87             passo. */
88     }
89 }
90 double elapsed = agora() - start;
91 double peak = u[0], minimum = u[0], sum = 0.0;
92 for (size_t c = 0; c < count; ++c) {
93     peak = fmax(peak, u[c]); minimum = fmin(minimum, u[c]); sum += u[c
94 ];
95 }

```

```

90     printf("%s tamanho_grid=%d num_passos_tempo=%d modo=%d threads=%d r
      =%.3f tempo=%.9f s minimo=%.17g pico=%.17g soma=%.17g\n",
91         "v3_dynamic", tamanho_grid, num_passos_tempo, mode, threads, r,
            elapsed, minimum, peak, sum);
92     int status = EXIT_SUCCESS;
93     if (argc == 3) {
94         FILE *file = fopen(argv[2], "w");
95         if (!file) { perror(argv[2]); status = EXIT_FAILURE; }
96         else {
97             for (size_t c = 0; c < count; ++c)
98                 if (fprintf(file, "%.17g%c", u[c], (c+1)%tamanho_grid ? '
                    ' : '\n') < 0)
99                     status = EXIT_FAILURE;
100             if (fclose(file)) status = EXIT_FAILURE;
101         }
102     }
103     free(u); free(v);
104     return status;
105 }

```

Listing 7: v3_dynamic.c: dynamic por linhas

```

1 // gcc -std=c11 -O2 -Wall -Wextra -fopenmp v4_dynamic_collapse.c -lm -o
   ns_v4
2 #ifndef _OPENMP
3 #error "Compile esta variante com -fopenmp"
4 #endif
5 #define _POSIX_C_SOURCE 199309L
6 #ifndef CHUNK_SIZE
7 #define CHUNK_SIZE 16
8 #endif
9 #include <errno.h>
10 #include <math.h>
11 #include <stdio.h>
12 #include <stdlib.h>
13 #include <string.h>
14 #include <time.h>
15 #include <omp.h>
16
17 static int integer(const char *text, int minimum, int maximum, int *out) {
18     char *end;
19     errno = 0;
20     long value = strtol(text, &end, 10);
21     if (errno || end == text || *end || value < minimum || value > maximum
22         )
23         return 0;
24     *out = (int)value;
25     return 1;
26 }
27
28 static double agora(void) {
29     struct timespec t;
30     if (clock_gettime(CLOCK_MONOTONIC, &t)) {
31         perror("clock_gettime");
32         exit(EXIT_FAILURE);
33     }
34     return (double)t.tv_sec + (double)t.tv_nsec / 1e9;
35 }
36
37 int main(int argc, char **argv) {
38     const int tamanho_grid = 512;
39     const int num_passos_tempo = 500;
40     int mode = 0, threads = 1;
41     if (argc > 3 || (argc > 1 && !integer(argv[1], 0, 2, &mode))) {
42         fprintf(stderr, "Uso: %s [modo=0|1|2 [campo.txt]]\n", argv[0]);
43         return EXIT_FAILURE;
44     }
45     size_t count = (size_t)tamanho_grid * tamanho_grid;
46     double *u = malloc(count * sizeof(*u));
47     double *v = malloc(count * sizeof(*v));

```

```

47 if (!u || !v) {
48     fprintf(stderr, "Falha de alocação\n");
49     free(u); free(v);
50     return EXIT_FAILURE;
51 }
52 const double nu = 0.1, dt = 0.2, dx = 1.0;
53 const double r = nu * dt / (dx * dx); /* 0.02 <= 1/4 em 2D */
54 for (int i = 0; i < tamanho_grid; ++i) {
55     for (int j = 0; j < tamanho_grid; ++j) {
56         double x = i - (tamanho_grid - 1) / 2.0, y = j - (tamanho_grid
57             - 1) / 2.0;
58         double sigma = tamanho_grid / 10.0;
59         int boundary = i == 0 || j == 0 || i == tamanho_grid-1 || j ==
60             tamanho_grid-1;
61         u[(size_t)i*tamanho_grid+j] = mode == 2 ? 1.0 :
62             (mode == 1 || boundary ? 0.0 :
63                 0.1 * exp(-(x*x+y*y)/(2*sigma*sigma)));
64     }
65 }
66 /* Bordas fixas iguais nos dois buffers: nenhuma escrita posterior. */
67 memcpy(v, u, count * sizeof(*v));
68 double start = agora();
69 #pragma omp parallel default(none) shared(u,v,tamanho_grid,
70     num_passos_tempo,r,threads)
71 {
72     #pragma omp single
73     threads = omp_get_num_threads();
74     for (int t = 0; t < num_passos_tempo; ++t) {
75         #pragma omp for collapse(2) schedule(dynamic, CHUNK_SIZE)
76         for (int i = 1; i < tamanho_grid-1; ++i) {
77             for (int j = 1; j < tamanho_grid-1; ++j) {
78                 size_t c = (size_t)i*tamanho_grid+j;
79                 v[c] = u[c] + r * (u[c-1]+u[c+1]+u[c-tamanho_grid]+u[c
80                     +tamanho_grid]-4*u[c]);
81             }
82         } /* Barreira: todas as escritas terminam antes da troca. */
83         #pragma omp single
84         {
85             double *tmp = u; u = v; v = tmp;
86         } /* Barreira: todas as threads veem a troca antes do próximo
87             passo. */
88     }
89 }
90 double elapsed = agora() - start;
91 double peak = u[0], minimum = u[0], sum = 0.0;
92 for (size_t c = 0; c < count; ++c) {
93     peak = fmax(peak, u[c]); minimum = fmin(minimum, u[c]); sum += u[c
94 ];
95 }

```

```

90     printf("%s tamanho_grid=%d num_passos_tempo=%d modo=%d threads=%d r
      =%.3f tempo=%.9f s minimo=%.17g pico=%.17g soma=%.17g\n",
91         "v4_dynamic_collapse", tamanho_grid, num_passos_tempo, mode,
          threads, r, elapsed, minimum, peak, sum);
92     int status = EXIT_SUCCESS;
93     if (argc == 3) {
94         FILE *file = fopen(argv[2], "w");
95         if (!file) { perror(argv[2]); status = EXIT_FAILURE; }
96         else {
97             for (size_t c = 0; c < count; ++c)
98                 if (fprintf(file, "%.17g%c", u[c], (c+1)%tamanho_grid ? '
          ' : '\n') < 0)
99                     status = EXIT_FAILURE;
100             if (fclose(file)) status = EXIT_FAILURE;
101         }
102     }
103     free(u); free(v);
104     return status;
105 }

```

Listing 8: v4_dynamic_collapse.c: dynamic por células

```

1 // gcc -std=c11 -O2 -Wall -Wextra -fopenmp v5_guided_collapse.c -lm -o
   ns_v5
2 #ifndef _OPENMP
3 #error "Compile esta variante com -fopenmp"
4 #endif
5 #define _POSIX_C_SOURCE 199309L
6 #ifndef CHUNK_SIZE
7 #define CHUNK_SIZE 16
8 #endif
9 #include <errno.h>
10 #include <math.h>
11 #include <stdio.h>
12 #include <stdlib.h>
13 #include <string.h>
14 #include <time.h>
15 #include <omp.h>
16
17 static int integer(const char *text, int minimum, int maximum, int *out) {
18     char *end;
19     errno = 0;
20     long value = strtol(text, &end, 10);
21     if (errno || end == text || *end || value < minimum || value > maximum
22         )
23         return 0;
24     *out = (int)value;
25     return 1;
26 }
27
28 static double agora(void) {
29     struct timespec t;
30     if (clock_gettime(CLOCK_MONOTONIC, &t)) {
31         perror("clock_gettime");
32         exit(EXIT_FAILURE);
33     }
34     return (double)t.tv_sec + (double)t.tv_nsec / 1e9;
35 }
36
37 int main(int argc, char **argv) {
38     const int tamanho_grid = 512;
39     const int num_passos_tempo = 500;
40     int mode = 0, threads = 1;
41     if (argc > 3 || (argc > 1 && !integer(argv[1], 0, 2, &mode))) {
42         fprintf(stderr, "Uso: %s [modo=0|1|2 [campo.txt]]\n", argv[0]);
43         return EXIT_FAILURE;
44     }
45     size_t count = (size_t)tamanho_grid * tamanho_grid;
46     double *u = malloc(count * sizeof(*u));
47     double *v = malloc(count * sizeof(*v));

```

```

47 if (!u || !v) {
48     fprintf(stderr, "Falha de alocação\n");
49     free(u); free(v);
50     return EXIT_FAILURE;
51 }
52 const double nu = 0.1, dt = 0.2, dx = 1.0;
53 const double r = nu * dt / (dx * dx); /* 0.02 <= 1/4 em 2D */
54 for (int i = 0; i < tamanho_grid; ++i) {
55     for (int j = 0; j < tamanho_grid; ++j) {
56         double x = i - (tamanho_grid - 1) / 2.0, y = j - (tamanho_grid
57             - 1) / 2.0;
58         double sigma = tamanho_grid / 10.0;
59         int boundary = i == 0 || j == 0 || i == tamanho_grid-1 || j ==
60             tamanho_grid-1;
61         u[(size_t)i*tamanho_grid+j] = mode == 2 ? 1.0 :
62             (mode == 1 || boundary ? 0.0 :
63                 0.1 * exp(-(x*x+y*y)/(2*sigma*sigma)));
64     }
65 }
66 /* Bordas fixas iguais nos dois buffers: nenhuma escrita posterior. */
67 memcpy(v, u, count * sizeof(*v));
68 double start = agora();
69 #pragma omp parallel default(none) shared(u,v,tamanho_grid,
70     num_passos_tempo,r,threads)
71 {
72     #pragma omp single
73     threads = omp_get_num_threads();
74     for (int t = 0; t < num_passos_tempo; ++t) {
75         #pragma omp for collapse(2) schedule(guided, CHUNK_SIZE)
76         for (int i = 1; i < tamanho_grid-1; ++i) {
77             for (int j = 1; j < tamanho_grid-1; ++j) {
78                 size_t c = (size_t)i*tamanho_grid+j;
79                 v[c] = u[c] + r * (u[c-1]+u[c+1]+u[c-tamanho_grid]+u[c
80                     +tamanho_grid]-4*u[c]);
81             }
82         } /* Barreira: todas as escritas terminam antes da troca. */
83         #pragma omp single
84         {
85             double *tmp = u; u = v; v = tmp;
86         } /* Barreira: todas as threads veem a troca antes do próximo
87             passo. */
88     }
89 }
90 double elapsed = agora() - start;
91 double peak = u[0], minimum = u[0], sum = 0.0;
92 for (size_t c = 0; c < count; ++c) {
93     peak = fmax(peak, u[c]); minimum = fmin(minimum, u[c]); sum += u[c
94 ];
95 }

```

```

90     printf("%s tamanho_grid=%d num_passos_tempo=%d modo=%d threads=%d r
      =%.3f tempo=%.9f s minimo=%.17g pico=%.17g soma=%.17g\n",
91         "v5_guided_collapse", tamanho_grid, num_passos_tempo, mode,
          threads, r, elapsed, minimum, peak, sum);
92     int status = EXIT_SUCCESS;
93     if (argc == 3) {
94         FILE *file = fopen(argv[2], "w");
95         if (!file) { perror(argv[2]); status = EXIT_FAILURE; }
96         else {
97             for (size_t c = 0; c < count; ++c)
98                 if (fprintf(file, "%.17g%c", u[c], (c+1)%tamanho_grid ? '
          ' : '\n') < 0)
99                     status = EXIT_FAILURE;
100             if (fclose(file)) status = EXIT_FAILURE;
101         }
102     }
103     free(u); free(v);
104     return status;
105 }

```

Listing 9: v5_guided_collapse.c: guided por células

```

1 // gcc -std=c11 -O2 -Wall -Wextra -fopenmp v6_guided.c -lm -o ns_v6
2 #ifndef _OPENMP
3 #error "Compile esta variante com -fopenmp"
4 #endif
5 #define _POSIX_C_SOURCE 199309L
6 #ifndef CHUNK_SIZE
7 #define CHUNK_SIZE 16
8 #endif
9 #include <errno.h>
10 #include <math.h>
11 #include <stdio.h>
12 #include <stdlib.h>
13 #include <string.h>
14 #include <time.h>
15 #include <omp.h>
16
17 static int integer(const char *text, int minimum, int maximum, int *out) {
18     char *end;
19     errno = 0;
20     long value = strtol(text, &end, 10);
21     if (errno || end == text || *end || value < minimum || value > maximum
22         )
23         return 0;
24     *out = (int)value;
25     return 1;
26 }
27
28 static double agora(void) {
29     struct timespec t;
30     if (clock_gettime(CLOCK_MONOTONIC, &t)) {
31         perror("clock_gettime");
32         exit(EXIT_FAILURE);
33     }
34     return (double)t.tv_sec + (double)t.tv_nsec / 1e9;
35 }
36
37 int main(int argc, char **argv) {
38     const int tamanho_grid = 512;
39     const int num_passos_tempo = 500;
40     int mode = 0, threads = 1;
41     if (argc > 3 || (argc > 1 && !integer(argv[1], 0, 2, &mode))) {
42         fprintf(stderr, "Uso: %s [modo=0|1|2 [campo.txt]]\n", argv[0]);
43         return EXIT_FAILURE;
44     }
45     size_t count = (size_t)tamanho_grid * tamanho_grid;
46     double *u = malloc(count * sizeof(*u));
47     double *v = malloc(count * sizeof(*v));
48     if (!u || !v) {

```

```

48     fprintf(stderr, "Falha de alocação\n");
49     free(u); free(v);
50     return EXIT_FAILURE;
51 }
52 const double nu = 0.1, dt = 0.2, dx = 1.0;
53 const double r = nu * dt / (dx * dx); /* 0.02 <= 1/4 em 2D */
54 for (int i = 0; i < tamanho_grid; ++i) {
55     for (int j = 0; j < tamanho_grid; ++j) {
56         double x = i - (tamanho_grid - 1) / 2.0, y = j - (tamanho_grid
57             - 1) / 2.0;
58         double sigma = tamanho_grid / 10.0;
59         int boundary = i == 0 || j == 0 || i == tamanho_grid-1 || j ==
60             tamanho_grid-1;
61         u[(size_t)i*tamanho_grid+j] = mode == 2 ? 1.0 :
62             (mode == 1 || boundary ? 0.0 :
63                 0.1 * exp(-(x*x+y*y)/(2*sigma*sigma)));
64     }
65 }
66 /* Bordas fixas iguais nos dois buffers: nenhuma escrita posterior. */
67 memcpy(v, u, count * sizeof(*v));
68 double start = agora();
69 #pragma omp parallel default(none) shared(u,v,tamanho_grid,
70     num_passos_tempo,r,threads)
71 {
72     #pragma omp single
73     threads = omp_get_num_threads();
74     for (int t = 0; t < num_passos_tempo; ++t) {
75         #pragma omp for schedule(guided, CHUNK_SIZE)
76         for (int i = 1; i < tamanho_grid-1; ++i) {
77             for (int j = 1; j < tamanho_grid-1; ++j) {
78                 size_t c = (size_t)i*tamanho_grid+j;
79                 v[c] = u[c] + r * (u[c-1]+u[c+1]+u[c-tamanho_grid]+u[c
80                     +tamanho_grid]-4*u[c]);
81             }
82         } /* Barreira: todas as escritas terminam antes da troca. */
83         #pragma omp single
84         {
85             double *tmp = u; u = v; v = tmp;
86         } /* Barreira: todas as threads veem a troca antes do próximo
87             passo. */
88     }
89 }
90 double elapsed = agora() - start;
91 double peak = u[0], minimum = u[0], sum = 0.0;
92 for (size_t c = 0; c < count; ++c) {
93     peak = fmax(peak, u[c]); minimum = fmin(minimum, u[c]); sum += u[c
94 ];
95 }

```

```

90     printf("%s tamanho_grid=%d num_passos_tempo=%d modo=%d threads=%d r
91           =%.3f tempo=%.9f s minimo=%.17g pico=%.17g soma=%.17g\n",
           "v6_guided", tamanho_grid, num_passos_tempo, mode, threads, r,
           elapsed, minimum, peak, sum);
92     int status = EXIT_SUCCESS;
93     if (argc == 3) {
94         FILE *file = fopen(argv[2], "w");
95         if (!file) { perror(argv[2]); status = EXIT_FAILURE; }
96         else {
97             for (size_t c = 0; c < count; ++c)
98                 if (fprintf(file, "%.17g%c", u[c], (c+1)%tamanho_grid ? '
99                     ' : '\n') < 0)
100                     status = EXIT_FAILURE;
101             if (fclose(file)) status = EXIT_FAILURE;
102         }
103     }
104     free(u); free(v);
105     return status;
}

```

Listing 10: v6_guided.c: guided por linhas